HiAgent 平台 API 调用示例

本文适用于 V1.4.x 及以后版本。

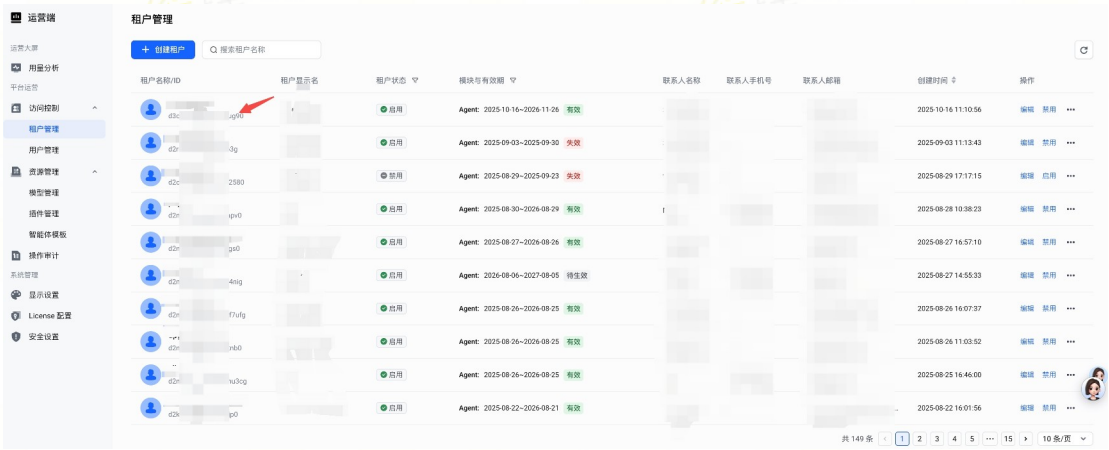
调用 API 需要获取如下参数：

1. Action：接口名
1. Version：固定为 `2023-08-01`
1. 请求地址
    1. TopHost：top 服务地址（根据部署情况用户自行配置 top 服务对外访问地址）

```Python
# kubectl get svc -n vke-system      volc-top -owide
NAME        TYPE        CLUSTER-IP       EXTERNAL-IP    PORT(S)
AGE     SELECTOR
volc-top    NodePort    xxx    <none>
8000:30040/TCP,56790:30041/TCP    268d    app=volc-
top,release=volc-top
```

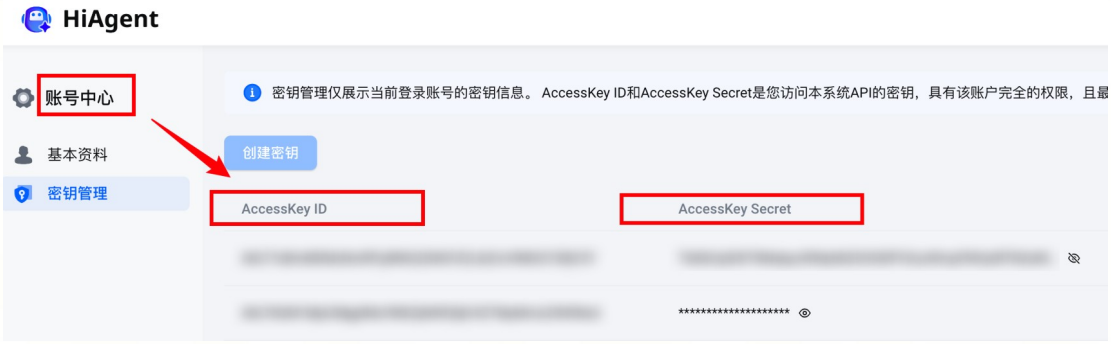
> `TopHost` 地址解释：hiagent 环境会部署 volc-top 服务，端口地址为 30040，外部用户一般不能直接访问，需要环境对应的运维人员暴露这个地址用于 API 调用

1. 主账号
    1. AccountID：默认值一般为 `1000000000`，可以在运营端->租户管理获取



1. 签名（请求过网关需要签名，这里 sign 参数会生成签名附带在请求头 `Authorization` 中）

1. AccessKeyID、SecretAccessKey：通过 点击头像 -> 账号中心 -> 密钥管理 获取



1. Region：区域（没特殊要求就用这个:`cn-north-1`）
1. Service：服务名，取值如下
    1.aigw: ai 网关
    1.app: 智能体
    1.eva: 评测
    1.iam: iam
    1.observe: 观测
    1.up: 文件上传
1. Version：对应服务接口版本，取值如下
    1.aigw version: 2023-08-01
    1.app version: 2023-08-01
    1.eva version: 2025-02-01
    1.iam version: 2024-12-25
    1.observe version: 2025-05-01
    1.up version: 2022-01-01

Golang

```Go
package top

import (
        "bytes"
        "context"
        "encoding/json"
        "fmt"
        "io"
        "net/http"
```

```
        "strings"
        "time"

        "github.com/volcengine/volc-sdk-golang/base"
)

var (
        topHost   = "{{$TopHost}}"
        accountID = {{$AccountID}}
        signer    = &base.Credentials{
                AccessKeyID:     "{{$AccessKeyID}}",     // 点击头像 -> 账号中心 -> 密钥管理
                SecretAccessKey: "{{$SecretAccessKey}}", // 点击头像 -> 账号中心 -> 密钥管理
                Region:          "{{$Region}}",          // 默认，没特殊要求就用这个:cn-north-1
                Service:         "{{$ServiceName}}",     // 调用服务名
        }
        httpTimeOut = 5 * time.Second
)

func Request(action, version string, body any, query string) ([]byte, error) {
        requesturl := fmt.Sprintf("%s?Action=%s&Version=%s&X-Account-Id=%d", topHost, action, version, accountID)
        if query != "" {
                if !strings.HasPrefix(query, "/") {
                        query = "&" + query
                }
                requesturl += query
        }
        b, _ := json.Marshal(body)

        req, _ := http.NewRequestWithContext(context.Background(), http.MethodPost, requesturl, bytes.NewBuffer(b))
        req.Header.Set("Content-Type", "application/json")

        req = signer.Sign(req)
        httpClient := &http.Client{
                Timeout: httpTimeOut,
        }

        /* do request and get return */
```

```
        rawResp, err := httpClient.Do(req)
        if err != nil {
                return nil, err
        } else {
                resp, err := io.ReadAll(rawResp.Body)
                if err != nil {
                        return nil, err
                }

                code := rawResp.StatusCode

                if code != http.StatusOK {
                        return nil, fmt.Errorf(string(resp))
                }
                return resp, nil
        }
}

func main(){
    resp, err := Request("{{ActionName}}", "{{Version}}",
"{{$body}}", "{{$query}}")
    fmt.Println(string(resp), err)
}
```

Java

推荐通过 Maven 依赖使用火山引擎 SDK for Java：详情见：火山引擎 SDK for Java

```
Java
<dependency>
    <groupId>org.apache.httpcomponents</groupId>
    <artifactId>httpclient</artifactId>
    <version>4.5.8</version>
</dependency>
<dependency>
    <groupId>com.volcengine</groupId>
    <artifactId>volc-sdk-java</artifactId>
    <version>1.0.49</version>
</dependency>
```

```
Java
import com.volcengine.auth.ISignerV4;
```

```
import com.volcengine.auth.impl.SignerV4Impl;
import com.volcengine.model.Credentials;
import com.volcengine.service.SignableRequest;
import org.apache.http.HttpEntity;
import org.apache.http.HttpResponse;
import org.apache.http.client.config.RequestConfig;
import org.apache.http.client.utils.URIBuilder;
import org.apache.http.entity.ByteArrayEntity;
import org.apache.http.entity.ContentType;
import org.apache.http.client.HttpClient;
import org.apache.http.impl.client.HttpClientBuilder;
import org.apache.http.util.Asserts;
import org.apache.http.util.EntityUtils;

public class RequestHandler {

    private ISignerV4 singer;
    private Credentials credentials = new Credentials();
    private HttpClient httpClient;
    private String url;

    // 这里ak(AccessKeyID)、sk(SecretAccessKey)、host(TopHost)根据
实际情况获取
    public void GenRequest(String ak, String sk, String host,
String action, String version) {
        singer = new SignerV4Impl();
        RequestConfig requestConfig =
RequestConfig.custom().setConnectionRequestTimeout(1000)
                .setSocketTimeout(3000).setConnectTimeout(1000).bu
ild();
        httpClient =
HttpClientBuilder.create().setDefaultRequestConfig(requestConfig).
build();

        credentials.setAccessKeyID(ak);
        credentials.setSecretAccessKey(sk);
        credentials.setRegion("cn-north-1");
        credentials.setService("app");

        url = host + "/?Action=" + action + "&Version=" + version
+ "&X-Account-Id=1000000000";
    }

    // 请求数据
```

```
    public void sendByteData(byte[] data) throws Exception {
        send(data, ContentType.APPLICATION_JSON);
    }

    private void send(byte[] data, ContentType contentType) throws
Exception {
        SignableRequest signableRequest;
        signableRequest = new SignableRequest();
        signableRequest.setMethod("POST");
        URIBuilder uriBuilder = new URIBuilder(url);

        signableRequest.setHeader("Content-Type",
contentType.getMimeType());

        signableRequest.setEntity(new ByteArrayEntity(data,
contentType));
        signableRequest.setUriBuilder(uriBuilder);
        signableRequest.setURI(uriBuilder.build());

        singer.sign(signableRequest, credentials);

        Asserts.notNull(signableRequest, "signableRequest can not
null");

        HttpResponse response =
httpClient.execute(signableRequest);

        if (response.getStatusLine().getStatusCode() != 200) {
            throw new Exception("http status code error");
        }

        HttpEntity respEntity = response.getEntity();
        byte[] content = EntityUtils.toByteArray(respEntity);

        // 自己处理输出
    }
}
```

Python

```
Go
pip install volcengine
```

```Python
from volcengine.auth.SignerV4 import SignerV4
from volcengine.auth.SignParam import SignParam
from volcengine.Credentials import Credentials
from collections import OrderedDict
import requests

method = 'POST'
host = '$HOST'
action = '$Action'
version = '2023-08-01'
ak = '$AK'
sk = '$SK'
region = 'cn-north-1'
service = 'app'

if __name__ == '__main__':
    sign = SignerV4()

    param = SignParam()
    param.path = '/'
    param.method = method
    param.host = host
    param.body = ''

    query = OrderedDict()
    query['Action'] = action
    query['Version'] = version
    param.query = query

    header = OrderedDict()
    header['Host'] = host
    param.header_list = header

    cren = Credentials(ak, sk, service, region)
    resulturl = sign.sign_url(param, cren)
    response = requests.request(method, url="http://" + param.host
+ '/?' + resulturl)

    print(response.text)
```

签名方式二（无 Action&Version 模式，智能体对话接口用）

```
Python
from volcengine.auth.SignerV4 import SignerV4
from volcengine.auth.SignParam import SignParam
from volcengine.Credentials import Credentials
from collections import OrderedDict
import requests

method = 'POST'
host = '$HOST'
ak = '$AK'
sk = '$SK'
region = 'cn-north-1'
service = 'app'

if __name__ == '__main__':
    sign = SignerV4()

    param = SignParam()
    param.path = '/'
    param.method = method
    param.host = host
    param.body = ''

    header = OrderedDict()
    header['Content-Type'] = 'application/json'
    param.headers = header

    cren = Credentials(ak, sk, service, region)
    sign.sign(param, cren)
    response = requests.request(method, url=param.host+param.path,
headers=param.headers, data=param.body)

    print(response.text)
```

代码调用示例

Java 调用示例

```
Java
代码块
import com.volcengine.auth.ISignerV4;
import com.volcengine.auth.impl.SignerV4Impl;
import com.volcengine.model.Credentials;
import com.volcengine.service.SignableRequest;
import org.apache.http.HttpEntity;
```

```
import org.apache.http.HttpResponse;
import org.apache.http.client.config.RequestConfig;
import org.apache.http.client.utils.URIBuilder;
import org.apache.http.entity.ByteArrayEntity;
import org.apache.http.entity.ContentType;
import org.apache.http.client.HttpClient;
import org.apache.http.impl.client.HttpClientBuilder;
import org.apache.http.util.Asserts;
import org.apache.http.util.EntityUtils;

public class RequestHandler {

    private ISignerV4 singer;
    private Credentials credentials = new Credentials();
    private HttpClient httpClient;
    private String url;

    // 这里ak(AccessKeyID)、sk(SecretAccessKey)、host(TopHost)根据
实际情况获取
    public void GenRequest(String ak, String sk, String host,
String action, String version) {
        singer = new SignerV4Impl();
        RequestConfig requestConfig =
RequestConfig.custom().setConnectionRequestTimeout(1000)
                .setSocketTimeout(3000).setConnectTimeout(1000).bu
ild();
        httpClient =
HttpClientBuilder.create().setDefaultRequestConfig(requestConfig).
build();

        credentials.setAccessKeyID(ak);
        credentials.setSecretAccessKey(sk);
        credentials.setRegion("cn-north-1");
        credentials.setService("app");

        url = host + "/?Action=" + action + "&Version=" + version
+ "&X-Account-Id=1000000000";
    }

    // 请求数据
    public void sendByteData(byte[] data) throws Exception {
        send(data, ContentType.APPLICATION_JSON);
    }
```

```
    private void send(byte[] data, ContentType contentType) throws
Exception {
        SignableRequest signableRequest;
        signableRequest = new SignableRequest();
        signableRequest.setMethod("POST");
        URIBuilder uriBuilder = new URIBuilder(url);

        signableRequest.setHeader("Content-Type",
contentType.getMimeType());

        signableRequest.setEntity(new ByteArrayEntity(data,
contentType));
        signableRequest.setUriBuilder(uriBuilder);
        signableRequest.setURI(uriBuilder.build());

        singer.sign(signableRequest, credentials);

        Asserts.notNull(signableRequest, "signableRequest can not
null");

        HttpResponse response =
httpClient.execute(signableRequest);

        if (response.getStatusLine().getStatusCode() != 200) {
            throw new Exception("http status code error");
        }

        HttpEntity respEntity = response.getEntity();
        byte[] content = EntityUtils.toByteArray(respEntity);

        // 自己处理输出
        // 将字节数组转换为字符串
        String answer = new String(content, "UTF-8");
        System.out.println(answer);
    }

    public static void main(String[] args) throws Exception {
        RequestHandler h = new RequestHandler();
        h.GenRequest("{AK}", "{SK}", "http://x.x.x.x:30040",
"GetTenantUsageStatistic", "2023-08-01");
        h.sendByteData("{\"TenantID\":
\"1000000000\"}".getBytes());
    }
}
```

```
##############响应内容##############
{
    "ResponseMetadata": {
        "RequestId":
"021733391315440000000000000000000ffffac15559dbd96f2",
        "Action": "GetTenantUsageStatistic",
        "Version": "2023-08-01",
        "Service": "app",
        "Region": "cn-north-1"
    },
    "Result": {
        "AppCount": 724,
        "UserCount": 6379,
        "TokenCount": 8837113,
        "WorkflowCount": 115,
        "WorkspaceCount": 138,
        "BaseResp": null
    }
}
```

Python 调用示例

```
Python
代码块
import json

from volcengine.auth.SignerV4 import SignerV4
from volcengine.auth.SignParam import SignParam
from volcengine.Credentials import Credentials
from collections import OrderedDict
import requests

method = 'POST'

action = 'ListAppCenter'
version = '2023-08-01'
host = "x.x.x.x:30040"  # 替换为实际的主机地址，top 的端口默认为:
30040，特殊情况再定
ak = "{AK}"  # 替换为实际的 AK，到用户端-账号中心-秘钥管理里获取
sk = "{SK}"  # 替换为实际的 SK，到用户端-账号中心-秘钥管理里获取
region = 'cn-north-1'
```

```
service = 'app'
if __name__ == '__main__':

    sign = SignerV4()

    headers = {
        'Content-Type': 'application/json'
    }

    param = SignParam()
    param.path = '/'
    param.method = method
    param.host = host
    param.body = ''

    query = OrderedDict()
    query['Action'] = action
    query['Version'] = version
    param.query = query

    header = OrderedDict()
    header['Host'] = host
    param.header_list = header

    data = {
        "App": True
    }

    cren = Credentials(ak, sk, service, region)
    resulturl = sign.sign_url(param, cren)
    response = requests.request(method, url="http://" + param.host
+ '/?' + resulturl, headers=headers, data=json.dumps(data))

    print(response.text)

###############响应内容###############
{
    "ResponseMetadata": {
        "RequestId":
"021733394053016000000000000000000ffff0a00007428f79f",
        "Action": "GetAppStatistic",
        "Version": "2023-08-01",
        "Service": "app",
        "Region": "cn-north-1"
```

```
    },
    "Result": {
        "Items": [....],
        "BaseResp": null
    }
}
```

方式二:

Go
代码块

```
import json

from volcengine.auth.SignerV4 import SignerV4
from volcengine.auth.SignParam import SignParam
from volcengine.Credentials import Credentials
from collections import OrderedDict
import requests

method = 'POST'
host = 'x.x.x.x:32300'
action = 'GetApiKey'
version = '2023-08-01'
ak = '{AK}'
sk = '{SK}'
region = 'cn-north-1'
service = 'app'

if __name__ == '__main__':
    sign = SignerV4()

    param = SignParam()
    param.path = '/api/proxy/api/v1/get_conversation_list'
    param.method = method
    param.host = host

    data = {
        "UserID": "2"
    }

    header = OrderedDict()
    header['Host'] = host
    header['Content-Type'] = 'application/json'
    header['X-APP-ID'] = 'cple9o61gima3osmo7gg'
```

```
    param.headers = header

    cren = Credentials(ak, sk, service, region)
    sign.sign(param, cren)
    response = requests.request(method, url="http://" +
param.host+param.path, headers=header, data=json.dumps(data))

    print(response.text)

    ##############响应内容##############
    {"ConversationList":[],"BaseResp":null}
```

Golang 调用示例

Go
代码块

```
package main

import (
   "bytes"
   "context"
   "encoding/json"
   "fmt"
   "io"
   "net/http"
   "strings"
   "time"

   "github.com/volcengine/volc-sdk-golang/base"
)

var (
   topHost   = "http://x.x.x.x:30040"
   accountID = 1000000000
   signer    = &base.Credentials{
      AccessKeyID:     "{AK}",              // 点击头像 -> 账号中心
-> 密钥管理
      SecretAccessKey: "{SK}", // 点击头像 -> 账号中心 -> 密钥管理
      Region:          "cn-north-1",
// 默认，没特殊要求就用这个:cn-north-1
      Service:         "app",
// 调用服务名
   }
   httpTimeOut = 5 * time.Second
)
```

```
func Request(action, version string, body any, query string)
([]byte, error) {
   requesturl := fmt.Sprintf("%s?Action=%s&Version=%s&X-Account-
Id=%d", topHost, action, version, accountID)
   if query != "" {
      if !strings.HasPrefix(query, "/") {
         query = "&" + query
      }
      requesturl += query
   }
   b, _ := json.Marshal(body)

   req, _ := http.NewRequestWithContext(context.Background(),
http.MethodPost, requesturl, bytes.NewBuffer(b))
   req.Header.Set("Content-Type", "application/json")

   req = signer.Sign(req)
   httpClient := &http.Client{
      Timeout: httpTimeOut,
   }

   /* do request and get return */
   rawResp, err := httpClient.Do(req)
   if err != nil {
      fmt.Println("打印错", err)
      return nil, err
   } else {
      resp, err := io.ReadAll(rawResp.Body)
      if err != nil {
         return nil, err
      }

      code := rawResp.StatusCode

      if code != http.StatusOK {
         return nil, fmt.Errorf(string(resp))
      }
      return resp, nil
   }
}

func main() {
   req := map[string]interface{}{
```

```
      "PageNumber": 1,
      "PageSize":   5,
   }
   resp, err := Request("GetAppMessageStatistic", "2023-08-01",
req, "")
   fmt.Println(string(resp), err)
}

###############响应内容###############
{
    "ResponseMetadata": {
        "RequestId":
"0217333946738930000000000000000000ffff0a000074b3c2f0",
        "Action": "GetAppMessageStatistic",
        "Version": "2023-08-01",
        "Service": "app",
        "Region": "cn-north-1"
    },
    "Result": {
        "MessageNum": 578,
        "BaseResp": null
    }
}
```

国密算法看这里

Python 调用示例

```
Python
import json
import requests
import datetime
from cryptography.hazmat.primitives import hashes, hmac
from urllib.parse import quote

class TopAPICallException(Exception):
    pass

class HTTPClient:
    def __init__(
        self, host: str, region: str, service: str, ak: str, sk:
str, use_sm: bool
    ) -> None:
```

```
        self.host = host
        self.region = region
        self.service = service
        self.ak = ak
        self.sk = sk
        self.use_sm = use_sm

    def post(self, action: str, version: str, req: dict,
tenant_id: str) -> str:
        headers = {
            "Content-Type": "application/json",
            "X-Tenant-Id": str(tenant_id),
            "X-Top-Tenant-Id": str(tenant_id),
        }
        query = {
            "Action": action,
            "Version": version,
            "X-Tenant-Id": str(tenant_id),
        }
        body = json.dumps(req)

        SignerV4.sign(
            path="/",
            method="POST",
            headers=headers,
            body=body,
            query=query,
            ak=self.ak,
            sk=self.sk,
            service=self.service,
            region=self.region,
            use_sm=self.use_sm,
        )

        resp = requests.post(
            url=self.host,
            headers=headers,
            params=query,
            data=body,
        )

        if resp.status_code >= 300:
            raise TopAPICallException(
                f"request error, action: {action}, status code:
```

```
{resp.status_code}, body: {resp.text}",
            )

        return resp.text

    def createWorkspace(self, req: dict, tenant_id: str) :
        resp = client.post(
                action="CreateWorkspaceForAdmin",
                version="2024-12-25",
                req=req,
                tenant_id=tenant_id,
            )
        print(resp)

    def deleteWorkspace(self, req: dict, tenant_id: str) :
        resp = client.post(
                action="DeleteWorkspace",
                version="2024-12-25",
                req=req,
                tenant_id=tenant_id,
            )
        print(resp)

class SignerV4(object):
    @staticmethod
    def sign(
        path,
        method,
        headers,
        body,
        query,
        ak,
        sk,
        region,
        service,
        use_sm: bool = False,
    ):
        if path == "":
            path = "/"
        if method != "GET" and not (
            "Content-Type" in headers or "content-type" in headers
        ):
            headers["Content-Type"] = "application/x-www-form-
urlencoded; charset=utf-8"
```

```
        format_date =
datetime.datetime.utcnow().strftime("%Y%m%dT%H%M%SZ")
        headers["X-Date"] = format_date

        body_hash = SignerV4.hashSum(body.encode("utf-8"),
use_sm).hex()
        headers["X-Content-Sha256"] = body_hash

        signed_headers = dict()
        for key in headers:
            if key in [
                "Content-Type",
                "content-type",
                "Content-Md5",
                "Host",
            ] or key.startswith("X-"):
                signed_headers[key.lower()] = headers[key]

        if "host" in signed_headers:
            v = signed_headers["host"]
            if v.find(":") != -1:
                split = v.split(":")
                port = split[1]
                if str(port) == "80" or str(port) == "443":
                    signed_headers["host"] = split[0]

        signed_str = ""
        for key in sorted(signed_headers.keys()):
            signed_str += key + ":" + signed_headers[key] + "\n"

        signed_headers_string =
";".join(sorted(signed_headers.keys()))

        canonical_request = "\n".join(
            [
                method,
                path,
                SignerV4.canonical_query(dict(query)),
                signed_str,
                signed_headers_string,
                body_hash,
            ]
        )
        credential_scope = "/".join([format_date[:8], region,
```

```
service, "request"])
        signing_str = "\n".join(
            [
                "HMAC-SHA256",
                format_date,
                credential_scope,

SignerV4.hashSum(canonical_request.encode("utf-8"), use_sm).hex(),
            ]
        )
        signing_key = SignerV4.get_signing_secret_key_v4(
            sk, format_date[:8], region, service, use_sm
        )

        signature = SignerV4.hmac_sum(
            signing_key, signing_str.encode("utf-8"), use_sm
        ).hex()

        credential = ak + "/" + credential_scope
        headers["Authorization"] = (
            "HMAC-SHA256"
            + " Credential="
            + credential
            + ", SignedHeaders="
            + signed_headers_string
            + ", Signature="
            + signature
        )
        return

    @staticmethod
    def canonical_query(query):
        res = []
        for key in query:
            value = str(query[key])
            res.append((quote(key, safe="-_.~"), quote(value,
safe="-_.~")))
        sorted_key_vals = []
        for key, value in sorted(res):
            sorted_key_vals.append("%s=%s" % (key, value))
        return "&".join(sorted_key_vals)

    @staticmethod
    def get_signing_secret_key_v4(sk, date, region, service,
```

```
use_sm: bool = False):
        kdate = SignerV4.hmac_sum(sk.encode("utf-8"),
date.encode("utf-8"), use_sm)
        kregion = SignerV4.hmac_sum(kdate, region.encode("utf-8"),
use_sm)
        kservice = SignerV4.hmac_sum(kregion,
service.encode("utf-8"), use_sm)
        return SignerV4.hmac_sum(kservice,
"request".encode("utf-8"), use_sm)

    @staticmethod
    def hmac_sum(key: bytes, msg: bytes, use_sm: bool = False):
        hasher = hashes.SHA256()
        if use_sm:
            hasher = hashes.SM3()
        h = hmac.HMAC(key, hasher)
        h.update(msg)
        return h.finalize()

    @staticmethod
    def hashSum(msg: bytes, use_sm: bool = False):
        hasher = hashes.SHA256()
        if use_sm:
            hasher = hashes.SM3()
        digest = hashes.Hash(hasher)
        digest.update(msg)
        return digest.finalize()

if __name__ == "__main__":
    client = HTTPClient(
        host="http://xxx:8000",  # top service 地址
        region="cn-north-1",
        service="iam",
        ak="xxx",  #ak
        sk="xxx",  # sk
        use_sm=True, #使用国密
    )

    client.createWorkspace(req={
            "Name": "workspace-system",
            "OwnerID": "d2jt3tkrnllh3up12qdg" #租户管理员 id
        }, tenant_id="d2jt3tkrnllh3up12qcg") #租户 id
```

Java 调用示例

```Java
package com.bytedance.eps.iam.server;

import com.google.gson.Gson;
import org.bouncycastle.jce.provider.BouncyCastleProvider;

import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.io.IOException;
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.security.InvalidKeyException;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.Security;
import java.time.ZoneOffset;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.util.stream.Collectors;

/**
 * This class is a Java translation of the provided Python script
for making signed API calls.
 * It includes an HTTP client, a SignerV4 implementation for
request signing, and an example usage in the main method.
 *
 * Note: This code requires the following dependencies:
 * - com.google.code.gson:gson (for JSON processing)
 * - org.bouncycastle:bcprov-jdk18on (for SM3 hash algorithm)
 */
public class Top {

    static {
        // Add Bouncy Castle provider for SM3 support
        Security.addProvider(new BouncyCastleProvider());
    }
```

```
    /**
     * Custom exception for API call errors.
     */
    public static class TopAPICallException extends Exception {
        public TopAPICallException(String message) {
            super(message);
        }
    }

    /**
     * HTTPClient for making signed requests to the Top API.
     */
    public static class HTTPClient {
        private final String host;
        private final String region;
        private final String service;
        private final String ak;
        private final String sk;
        private final boolean useSm;
        private final HttpClient httpClient;
        private final Gson gson = new Gson();

        /**
         * Constructor for HTTPClient.
         * @param host The API host.
         * @param region The service region.
         * @param service The service name.
         * @param ak The access key.
         * @param sk The secret key.
         * @param useSm Whether to use SM3 for signing.
         */
        public HTTPClient(String host, String region, String
service, String ak, String sk, boolean useSm) {
            this.host = host;
            this.region = region;
            this.service = service;
            this.ak = ak;
            this.sk = sk;
            this.useSm = useSm;
            this.httpClient = HttpClient.newHttpClient();
        }

        /**
```

```
         * Makes a POST request to the API.
         * @param action The API action.
         * @param version The API version.
         * @param req The request payload.
         * @param tenantId The tenant ID.
         * @return The response body as a string.
         * @throws TopAPICallException if the API call fails.
         */
        public String post(String action, String version,
Map<String, Object> req, String tenantId) throws
TopAPICallException {
            try {
                Map<String, String> headers = new TreeMap<>();
                headers.put("Content-Type", "application/json");
                headers.put("X-Tenant-Id", tenantId);
                headers.put("X-Top-Tenant-Id", tenantId);

                Map<String, String> query = new TreeMap<>();
                query.put("Action", action);
                query.put("Version", version);
                query.put("X-Tenant-Id", tenantId);

                String body = gson.toJson(req);

                SignerV4.sign(
                        "/",
                        "POST",
                        headers,
                        body,
                        query,
                        this.ak,
                        this.sk,
                        this.region,
                        this.service,
                        this.useSm
                );

                String queryString =
SignerV4.canonicalQuery(query);
                URI uri = URI.create(this.host + "?" +
queryString);

                HttpRequest.Builder httpRequestBuilder =
HttpRequest.newBuilder()
```

```
                        .uri(uri)
                        .POST(HttpRequest.BodyPublishers.ofString(
body, StandardCharsets.UTF_8));

                headers.forEach(httpRequestBuilder::header);

                HttpResponse<String> response =
httpClient.send(httpRequestBuilder.build(),
HttpResponse.BodyHandlers.ofString());

                if (response.statusCode() >= 300) {
                    throw new TopAPICallException(
                            String.format("request error, action:
%s, status code: %d, body: %s",
                                    action, response.statusCode(),
response.body())
                    );
                }

                return response.body();
            } catch (Exception e) {
                throw new TopAPICallException("Failed to make POST
request: " + e.getMessage());
            }
        }

        /**
         * Creates a workspace by calling the
"CreateWorkspaceForAdmin" action.
         * @param req The request payload.
         * @param tenantId The tenant ID.
         * @throws TopAPICallException if the API call fails.
         */
        public void createWorkspace(Map<String, Object> req,
String tenantId) throws TopAPICallException {
            String resp = this.post(
                    "CreateWorkspaceForAdmin",
                    "2024-12-25",
                    req,
                    tenantId
            );
            System.out.println(resp);
        }
```

```
        /**
         * Deletes a workspace by calling the "DeleteWorkspace"
action.
         * @param req The request payload.
         * @param tenantId The tenant ID.
         * @throws TopAPICallException if the API call fails.
         */
        public void deleteWorkspace(Map<String, Object> req,
String tenantId) throws TopAPICallException {
            String resp = this.post(
                    "DeleteWorkspace",
                    "2024-12-25",
                    req,
                    tenantId
            );
            System.out.println(resp);
        }
    }

    /**
     * Implements the signing process.
     */
    public static class SignerV4 {

        private static final DateTimeFormatter yyyyMMddTHHmmssZ =
DateTimeFormatter.ofPattern("yyyyMMdd'T'HHmmss'Z'");

        /**
         * Signs the request and adds the "Authorization" header.
         */
        public static void sign(String path, String method,
Map<String, String> headers, String body,
                                Map<String, String> query, String
ak, String sk, String region,
                                String service, boolean useSm)
throws NoSuchAlgorithmException, InvalidKeyException {

            if (path == null || path.isEmpty()) {
                path = "/";
            }
            if (!"GET".equals(method) &&
headers.keySet().stream().noneMatch(k ->
k.equalsIgnoreCase("Content-Type"))) {
                headers.put("Content-Type", "application/x-www-
```

```
form-urlencoded; charset=utf-8");
            }

            String format_date =
ZonedDateTime.now(ZoneOffset.UTC).format(yyyyMMddTHHmmssZ);
            headers.put("X-Date", format_date);

            String bodyHash =
bytesToHex(hashSum(body.getBytes(StandardCharsets.UTF_8), useSm));
            headers.put("X-Content-Sha256", bodyHash);

            Map<String, String> signedHeaders = new TreeMap<>();
            for (Map.Entry<String, String> entry :
headers.entrySet()) {
                String key = entry.getKey();
                if (key.equalsIgnoreCase("Content-Type") ||
                    key.equalsIgnoreCase("Content-Md5") ||
                    key.equalsIgnoreCase("Host") ||
                    key.toLowerCase().startsWith("x-")) {
                    signedHeaders.put(key.toLowerCase(),
entry.getValue());
                }
            }

            if (signedHeaders.containsKey("host")) {
                String v = signedHeaders.get("host");
                if (v.contains(":")) {
                    String[] split = v.split(":");
                    String port = split[1];
                    if ("80".equals(port) || "443".equals(port)) {
                        signedHeaders.put("host", split[0]);
                    }
                }
            }

            StringBuilder signedStrBuilder = new StringBuilder();
            for (Map.Entry<String, String> entry :
signedHeaders.entrySet()) {

signedStrBuilder.append(entry.getKey()).append(":").append(entry.g
etValue()).append("\n");
            }

            String signedHeadersString = String.join(";",
```

```
signedHeaders.keySet());

            String canonicalRequest = String.join("\n",
                    method,
                    path,
                    canonicalQuery(query),
                    signedStrBuilder.toString(),
                    signedHeadersString,
                    bodyHash
            );

            String credentialScope = String.join("/",
format_date.substring(0, 8), region, service, "request");
            String signingStr = String.join("\n",
                     "HMAC-SHA256",
                    format_date,
                    credentialScope,

bytesToHex(hashSum(canonicalRequest.getBytes(StandardCharsets.UTF_
8), useSm))
            );

            byte[] signingKey = getSigningSecretKeyV4(sk,
format_date.substring(0, 8), region, service, useSm);
            String signature = bytesToHex(hmacSum(signingKey,
signingStr.getBytes(StandardCharsets.UTF_8), useSm));

            String credential = ak + "/" + credentialScope;
            String authorization = ("HMAC-SHA256") +
                    " Credential=" + credential +
                    ", SignedHeaders=" + signedHeadersString +
                    ", Signature=" + signature;
            headers.put("Authorization", authorization);
        }

        /**
         * Creates a canonical query string from a map of query
parameters.
         */
        public static String canonicalQuery(Map<String, String>
query) {
            if (query == null || query.isEmpty()) {
                return "";
            }
```

```
            return query.entrySet().stream()
                    .sorted(Map.Entry.comparingByKey())
                    .map(e -> urlEncode(e.getKey()) + "=" +
urlEncode(e.getValue()))
                    .collect(Collectors.joining("&"));
        }

        private static String urlEncode(String value) {
            try {
                return URLEncoder.encode(value,
StandardCharsets.UTF_8.name())
                        .replace("+", "%20")
                        .replace("*", "%2A")
                        .replace("%7E", "~");
            } catch (Exception e) {
                throw new RuntimeException(e);
            }
        }

        /**
         * Derives the signing key for Signature Version 4.
         */
        private static byte[] getSigningSecretKeyV4(String sk,
String date, String region, String service, boolean useSm) throws
InvalidKeyException, NoSuchAlgorithmException {
            byte[] kDate =
hmacSum(sk.getBytes(StandardCharsets.UTF_8),
date.getBytes(StandardCharsets.UTF_8), useSm);
            byte[] kRegion = hmacSum(kDate,
region.getBytes(StandardCharsets.UTF_8), useSm);
            byte[] kService = hmacSum(kRegion,
service.getBytes(StandardCharsets.UTF_8), useSm);
            return hmacSum(kService,
"request".getBytes(StandardCharsets.UTF_8), useSm);
        }

        /**
         * Calculates HMAC signature.
         */
        private static byte[] hmacSum(byte[] key, byte[] msg,
boolean useSm) throws NoSuchAlgorithmException,
InvalidKeyException {
            String alg = useSm ? "HmacSM3" : "HmacSHA256";
            Mac mac = Mac.getInstance(alg);
```

```
            mac.init(new SecretKeySpec(key, alg));
            return mac.doFinal(msg);
        }

        /**
         * Calculates a hash (SHA-256 or SM3).
         */
        private static byte[] hashSum(byte[] msg, boolean useSm)
throws NoSuchAlgorithmException {
            String alg = useSm ? "SM3" : "SHA-256";
            MessageDigest digest = MessageDigest.getInstance(alg);
            return digest.digest(msg);
        }

        private static final char[] HEX_ARRAY =
"0123456789abcdef".toCharArray();

        /**
         * Converts a byte array to a hexadecimal string.
         */
        public static String bytesToHex(byte[] bytes) {
            char[] hexChars = new char[bytes.length * 2];
            for (int j = 0; j < bytes.length; j++) {
                int v = bytes[j] & 0xFF;
                hexChars[j * 2] = HEX_ARRAY[v >>> 4];
                hexChars[j * 2 + 1] = HEX_ARRAY[v & 0x0F];
            }
            return new String(hexChars);
        }
    }

    /**
     * Main method for demonstration.
     */
    public static void main(String[] args) {
        HTTPClient client = new HTTPClient(
                "http://127.0.0.1:8889", // top service address
                "cn-north-1",
                "iam",
                "test_ak", // ak
                "test_sk", // sk
                true // use SM3
        );
```

```
        try {
            Map<String, Object> req = new HashMap<>();
            req.put("Name", "workspace-system");
            req.put("OwnerID", "d2rh64ucckeegtm1ro70"); // tenant
admin id

            client.createWorkspace(req,
"d2rh64ucckeegtm1ro60"); // tenant id
        } catch (TopAPICallException e) {
            e.printStackTrace();
        }
    }
}
```

Golang 调用示例

Go

```
package main

import (
    "bytes"
    "context"
    "crypto/hmac"
    "encoding/hex"
    "encoding/json"
    "errors"
    "fmt"
    "hash"
    "io"
    "net"
    "net/http"
    "net/url"
    "sort"
    "strings"
    "time"

    "github.com/emmansun/gmsm/sm3"
)

const (
    APIVersion20241225 = "2024-12-25"
    RegionCnNorth1     = "cn-north-1"
    timeFormatV4       = "20060102T150405Z"
)
```

```
func main() {
    cli := NewClient(&ServiceInfo{
       Scheme: "http",
       Host:   <替换成 TOP 地址，比如 10.0.0.1:30040>,
       Credentials: Credentials{
          Service:   "iam",
          Region:    RegionCnNorth1,
          AccessKey: <替换成真实的 AK>,
          SecretKey: <替换成真实的 SK>}})
    respond, err :=
cli.CreateWorkspaceForAdmin(context.Background(),
&CreateWorkspaceRequest{
       Name:    "hl",
       OwnerID: "xxxxxxx",
    })

    if err != nil {
       fmt.Println(err)
       return
    }
    fmt.Println(respond.ID)
}

type CreateWorkspaceRequest struct {
    // 名称
    Name string
    // 空间所有人，传 IAM user id
    OwnerID string
}

type CreateWorkspaceResponse struct {
    ID string `thrift:"ID,1" json:"ID" `
}

type Credentials struct {
    AccessKey string
    SecretKey string
    Service   string
    Region    string
}

type ServiceInfo struct {
    Scheme      string
    Host        string
```

```
    Header      http.Header
    Credentials Credentials
}

// NewClient ...
func NewClient(info *ServiceInfo) Interface {
    transport := &http.Transport{
       DialContext: (&net.Dialer{
          Timeout:   5 * time.Second,  // 连接建立超时时间
          KeepAlive: 60 * time.Second, // TCP KeepAlive 时间
       }).DialContext,
       ForceAttemptHTTP2:     true,              // 尽可能尝试
HTTP/2
       MaxIdleConns:          1000,              // 最大空闲连接总数
       MaxIdleConnsPerHost:   100,               // 每个主机的最大空
闲连接数
       IdleConnTimeout:       90 * time.Second, // 空闲连接超时时间
(调整为90 秒)
       TLSHandshakeTimeout:   10 * time.Second, // TLS 握手超时
       ExpectContinueTimeout: 1 * time.Second,
    }

    c := &http.Client{Transport: transport}
    client := &Client{Client: c, ServiceInfo: info}

    if client.ServiceInfo.Scheme == "" {
       client.ServiceInfo.Scheme = "http"
    }
    if client.ServiceInfo.Host == "" {
       client.ServiceInfo.Host = "top-server:8000"
    }

    if client.ServiceInfo.Credentials.Region == "" {
       client.ServiceInfo.Credentials.Region = RegionCnNorth1
    }

    return client
}

type Client struct {
    Client       *http.Client
    ServiceInfo  *ServiceInfo
    IdentityInfo *IdentityInfo
}
```

```
type CheckSignatureResult struct {
    TenantID     string
    IdentityID   string
    IdentityType int
}

type Interface interface {
    CreateWorkspaceForAdmin(ctx context.Context, req
*CreateWorkspaceRequest) (r *CreateWorkspaceResponse, err error)
    SetUserID(userID string) Interface
    SetTenantID(tenantID string) Interface
    AddHeaders(headers http.Header) Interface
    SetLanguage(language string) Interface
}

type IdentityInfo struct {
    UserID   string
    TenantID string
}

// SetUserID IdentityInfo 只在单次请求中生效
func (c *Client) SetUserID(userID string) Interface {
    oneTimeCli := &Client{
       Client:       c.Client,
       ServiceInfo:  c.ServiceInfo,
       IdentityInfo: c.IdentityInfo,
    }
    if oneTimeCli.IdentityInfo == nil {
       oneTimeCli.IdentityInfo = &IdentityInfo{}
    }
    oneTimeCli.IdentityInfo.UserID = userID

    return oneTimeCli
}

// SetTenantID IdentityInfo 只在单次请求中生效
func (c *Client) SetTenantID(tenantID string) Interface {
    oneTimeCli := &Client{
       Client:       c.Client,
       ServiceInfo:  c.ServiceInfo,
       IdentityInfo: c.IdentityInfo,
    }
    if oneTimeCli.IdentityInfo == nil {
```

```
        oneTimeCli.IdentityInfo = &IdentityInfo{}
    }
    oneTimeCli.IdentityInfo.TenantID = tenantID

    return oneTimeCli
}

// AddHeaders 只在单次请求中生效
func (c *Client) AddHeaders(headers http.Header) Interface {
    oneTimeCli := &Client{
        Client:       c.Client,
        ServiceInfo:  &ServiceInfo{},
        IdentityInfo: c.IdentityInfo,
    }
    if oneTimeCli.IdentityInfo == nil {
        oneTimeCli.IdentityInfo = &IdentityInfo{}
    }
    oneTimeCli.ServiceInfo.Credentials = c.ServiceInfo.Credentials
    oneTimeCli.ServiceInfo.Host = c.ServiceInfo.Host
    oneTimeCli.ServiceInfo.Scheme = c.ServiceInfo.Scheme
    oneTimeCli.ServiceInfo.Header = make(http.Header)
    for k, v := range c.ServiceInfo.Header {
        oneTimeCli.ServiceInfo.Header[k] = v
    }
    for k, v := range headers {
        oneTimeCli.ServiceInfo.Header[k] = v
    }

    return oneTimeCli
}

func (c *Client) SetLanguage(language string) Interface {
    return c.AddHeaders(http.Header{
        "Accept-Language": []string{language},
    })
}

type ApiInfo struct {
    Method string
    Path   string
    Query  url.Values
    Header http.Header
}
```

```
func (c *Client) CreateWorkspace(ctx context.Context, req
*CreateWorkspaceRequest) (r *CreateWorkspaceResponse, err error) {
    r = &CreateWorkspaceResponse{}
    apiInfo := &ApiInfo{
       Method: http.MethodPost,
       Path:   "/",
       Query: url.Values{
          "Action":  []string{"CreateWorkspace"},
          "Version": []string{APIVersion20241225},
       },
    }
    body, err := json.Marshal(req)
    if err != nil {
       return nil, err
    }
    bytes, code, err := c.post(ctx, apiInfo, string(body))
    if err != nil || code != http.StatusOK {
       return nil, err
    }

    err = json.Unmarshal(bytes, r)
    if err != nil {
       return nil, err
    }
    return r, nil
}

func (c *Client) post(ctx context.Context, apiInfo *ApiInfo, body
string) ([]byte, int, error) {
    header := mergeHeader(c.ServiceInfo.Header, apiInfo.Header)
    header = addIdentityHeader(header, c.IdentityInfo)
    // Set to nil after using it once.
    c.IdentityInfo = nil
    query := apiInfo.Query.Encode()
    u := url.URL{
       Scheme:   c.ServiceInfo.Scheme,
       Host:     c.ServiceInfo.Host,
       Path:     apiInfo.Path,
       RawQuery: query,
    }
    req, err := http.NewRequest(strings.ToUpper(apiInfo.Method),
u.String(), strings.NewReader(body))
    if err != nil {
       return []byte(""), 500, errors.New("new http request
```

```
error")
    }
    req.Header = header
    req.Header.Set("Content-Type", "application/json")

    return c.makeRequest(ctx, req)
}

func addIdentityHeader(header http.Header, identityInfo
*IdentityInfo) http.Header {

    TenantID := "X-Tenant-Id"
    UserID := "X-User-Id"
    if identityInfo != nil {
       header.Set(TenantID, identityInfo.TenantID)
       header.Set(UserID, identityInfo.UserID)
    }

    return header
}

func (c *Client) CreateWorkspaceForAdmin(ctx context.Context, req
*CreateWorkspaceRequest) (r *CreateWorkspaceResponse, err error) {
    r = &CreateWorkspaceResponse{}
    apiInfo := &ApiInfo{
       Method: http.MethodPost,
       Path:   "/",
       Query: url.Values{
          "Action":  []string{"CreateWorkspaceForAdmin"},
          "Version": []string{APIVersion20241225},
       },
    }
    body, err := json.Marshal(req)
    if err != nil {
       return nil, err
    }
    bytes, code, err := c.post(ctx, apiInfo, string(body))
    if err != nil || code != http.StatusOK {
       return nil, err
    }

    err = json.Unmarshal(bytes, r)
    if err != nil {
       return nil, err
```

```
    }
    return r, nil
}

func (c *Client) makeRequest(ctx context.Context, req
*http.Request) ([]byte, int, error) {
    req = c.ServiceInfo.Credentials.Sign(req).WithContext(ctx)

    resp, err := c.Client.Do(req) // nolint
    if err != nil {
       return []byte(""), 500, err
    }
    defer func(Body io.ReadCloser) {
       err := Body.Close()
       if err != nil {
          print("failed to close response body")
       }
    }(resp.Body)
    body, err := io.ReadAll(resp.Body)
    if err != nil {
       return []byte(""), resp.StatusCode, err
    }

    if resp.StatusCode != http.StatusOK {
       return body, resp.StatusCode, fmt.Errorf("uri %s http code
%d body %s", req.RequestURI, resp.StatusCode, string(body))
    }

    bs, err := c.parseResult(body)
    if err != nil {
       return nil, resp.StatusCode, err
    }

    return bs, resp.StatusCode, nil
}

// ResponseMetadata ...
type ResponseMetadata struct {
    RequestID string     `json:"RequestId"`
    Action    string     `json:"Action"`
    Version   string     `json:"Version"`
    Service   string     `json:"Service"`
    Region    string     `json:"Region"`
    Error     *ErrorData `json:"Error,omitempty"`
```

```
}

// ErrorData ...
type ErrorData struct {
    HttpCode int32             `json:"HttpCode,omitempty"`
    Code     string            `json:"Code"`
    Message  string            `json:"Message"`
    Data     map[string]string `json:"Data"`
}

// Response ...
type Response struct {
    ResponseMetadata *ResponseMetadata `json:"ResponseMetadata"`
    Result           interface{}       `json:"Result,omitempty"`
}

func (c *Client) parseResult(body []byte) ([]byte, error) {
    respBody := &Response{}
    err := json.Unmarshal(body, respBody)
    if err != nil {
       return nil, err
    }
    res, err := json.Marshal(respBody.Result)
    if err != nil {
       return nil, err
    }
    return res, nil
}

func mergeHeader(header1, header2 http.Header) (header
http.Header) {
    header = http.Header{}
    for k, v := range header1 {
       header.Set(k, strings.Join(v, ";"))
    }

    for k, v := range header2 {
       header.Set(k, strings.Join(v, ";"))
    }

    return
}

func (c Credentials) Sign(request *http.Request) *http.Request {
```

```
    query := request.URL.Query()

    request.URL.RawQuery = query.Encode()
    return Sign4(request, c)
}

func (c Credentials) SignUrl(request *http.Request) string {
    query := request.URL.Query()
    ldt := timestampV4()
    sdt := ldt[:8]
    meta := new(metadata)
    meta.date, meta.service, meta.region, meta.signedHeaders,
meta.algorithm = sdt, c.Service, c.Region, "", "HMAC-SHA256"
    meta.credentialScope = concat("/", meta.date, meta.region,
meta.service, "request")

    query.Set("X-Date", ldt)
    query.Set("X-Credential",
c.AccessKey+"/"+meta.credentialScope)
    query.Set("X-Algorithm", meta.algorithm)
    query.Set("X-SignedHeaders", meta.signedHeaders)

    // Task 1
    hashedCanonReq := hashedSimpleCanonicalRequestV4(request,
query, meta)

    // Task 2
    stringToSign := concat("\n", meta.algorithm, ldt,
meta.credentialScope, hashedCanonReq)

    // Task 3
    signingKey := signingKeyV4(c.SecretKey, meta.date,
meta.region, meta.service)
    signature := signatureV4(signingKey, stringToSign)

    query.Set("X-Signature", signature)
    return query.Encode()
}

// Sign4 signs a request with Signed Signature Version 4.
func Sign4(request *http.Request, credential Credentials)
*http.Request {
    keys := credential
```

```
    prepareRequestV4(request)
    meta := new(metadata)
    meta.service, meta.region = keys.Service, keys.Region

    // Task 1
    hashedCanonReq := hashedCanonicalRequestV4(request, meta)

    // Task 2
    stringToSign := stringToSignV4(request, hashedCanonReq, meta)

    // Task 3
    signingKey := signingKeyV4(keys.SecretKey, meta.date,
meta.region, meta.service)
    signature := signatureV4(signingKey, stringToSign)

    request.Header.Set("Authorization",
buildAuthHeaderV4(signature, meta, keys))

    return request
}

func hashedSimpleCanonicalRequestV4(request *http.Request, query
url.Values, meta *metadata) string {
    payloadHash := hashSum([]byte(""))

    if request.URL.Path == "" {
       request.URL.Path = "/"
    }

    canonicalRequest := concat("\n", request.Method,
normuri(request.URL.Path), normquery(query), "\n",
meta.signedHeaders, payloadHash)

    return hashSum([]byte(canonicalRequest))
}

func hashedCanonicalRequestV4(request *http.Request, meta
*metadata) string {
    payload := readAndReplaceBody(request)
    payloadHash := hashSum(payload)
    request.Header.Set("X-Content-Sha256", payloadHash)

    request.Header.Set("Host", request.Host)
```

```
    var sortedHeaderKeys []string
    for key := range request.Header {
       switch key {
       case "Content-Type", "Content-Md5", "Host":
       default:
          if !strings.HasPrefix(key, "X-") {
             continue
          }
       }
       sortedHeaderKeys = append(sortedHeaderKeys,
strings.ToLower(key))
    }
    sort.Strings(sortedHeaderKeys)

    var headersToSign string
    for _, key := range sortedHeaderKeys {
       value := strings.TrimSpace(request.Header.Get(key))
       if key == "host" {
          if strings.Contains(value, ":") {
             split := strings.Split(value, ":")
             port := split[1]
             if port == "80" || port == "443" {
                value = split[0]
             }
          }
       }
       headersToSign += key + ":" + value + "\n"
    }
    meta.signedHeaders = concat(";", sortedHeaderKeys...)
    canonicalRequest := concat("\n", request.Method,
normuri(request.URL.Path), normquery(request.URL.Query()),
headersToSign, meta.signedHeaders, payloadHash)

    return hashSum([]byte(canonicalRequest))
}

func stringToSignV4(request *http.Request, hashedCanonReq string,
meta *metadata) string {
    requestTs := request.Header.Get("X-Date")

    meta.algorithm = "HMAC-SHA256"
    meta.date = tsDateV4(requestTs)
    meta.credentialScope = concat("/", meta.date, meta.region,
meta.service, "request")
```

```
    return concat("\n", meta.algorithm, requestTs,
meta.credentialScope, hashedCanonReq)
}

func signatureV4(signingKey []byte, stringToSign string) string {
    return hex.EncodeToString(hmacSign(signingKey, stringToSign))
}

func prepareRequestV4(request *http.Request) *http.Request {
    necessaryDefaults := map[string]string{
       "Content-Type": "application/x-www-form-urlencoded;
charset=utf-8",
       "X-Date":       timestampV4(),
    }

    for header, value := range necessaryDefaults {
       if request.Header.Get(header) == "" {
          request.Header.Set(header, value)
       }
    }

    if request.URL.Path == "" {
       request.URL.Path += "/"
    }

    return request
}

func signingKeyV4(secretKey, date, region, service string) []byte
{
    kDate := hmacSign([]byte(secretKey), date)
    kRegion := hmacSign(kDate, region)
    kService := hmacSign(kRegion, service)
    kSigning := hmacSign(kService, "request")
    return kSigning
}

func buildAuthHeaderV4(signature string, meta *metadata, keys
Credentials) string {
    credential := keys.AccessKey + "/" + meta.credentialScope

    return meta.algorithm +
       " Credential=" + credential +
```

```
       ", SignedHeaders=" + meta.signedHeaders +
       ", Signature=" + signature
}

func timestampV4() string {
    return now().Format(timeFormatV4)
}

func tsDateV4(timestamp string) string {
    return timestamp[:8]
}

func hmacSign(key []byte, content string) []byte {
    sm3alg := NewHmacSM3()
    sign2, err := sm3alg.Sign([]byte(content), string(key))
    if err != nil {
       print("hmacSign failed")
    }
    return sign2
}

func hashSum(content []byte) string {
    sum := sm3.Sum(content)
    return fmt.Sprintf("%x", sum)
}

func readAndReplaceBody(request *http.Request) []byte {
    if request.Body == nil {
       return []byte{}
    }
    payload, _ := io.ReadAll(request.Body)
    request.Body = io.NopCloser(bytes.NewReader(payload))
    return payload
}

func concat(delim string, str ...string) string {
    return strings.Join(str, delim)
}

var now = func() time.Time {
    return time.Now().UTC()
}

func normuri(uri string) string {
```

```
    parts := strings.Split(uri, "/")
    for i := range parts {
       parts[i] = encodePathFrag(parts[i])
    }
    return strings.Join(parts, "/")
}

func encodePathFrag(s string) string {
    hexCount := 0
    for i := 0; i < len(s); i++ {
       c := s[i]
       if shouldEscape(c) {
          hexCount++
       }
    }
    t := make([]byte, len(s)+2*hexCount)
    j := 0
    for i := 0; i < len(s); i++ {
       c := s[i]
       if shouldEscape(c) {
          t[j] = '%'
          t[j+1] = "0123456789ABCDEF"[c>>4]
          t[j+2] = "0123456789ABCDEF"[c&15]
          j += 3
       } else {
          t[j] = c
          j++
       }
    }
    return string(t)
}

func shouldEscape(c byte) bool {
    if 'a' <= c && c <= 'z' || 'A' <= c && c <= 'Z' {
       return false
    }
    if '0' <= c && c <= '9' {
       return false
    }
    if c == '-' || c == '_' || c == '.' || c == '~' {
       return false
    }
    return true
}
```

```
func normquery(v url.Values) string {
    queryString := v.Encode()

    return strings.ReplaceAll(queryString, "+", "%20")
}

type metadata struct {
    algorithm       string
    credentialScope string
    signedHeaders   string
    date            string
    region          string
    service         string
}

func NewHmacSM3() MacHasher {
    return &hmacCipher{
       macFactor:  hmac.New,
       hashFactor: sm3.New,
    }
}

type hmacCipher struct {
    macFactor  func(h func() hash.Hash, key []byte) hash.Hash
    hashFactor func() hash.Hash
}

func (a *hmacCipher) Sign(message []byte, key string) ([]byte,
error) {
    h := a.macFactor(a.hashFactor, []byte(key))
    h.Write([]byte(message))
    return h.Sum(nil), nil
}

// mac 签名接口
type MacHasher interface {
    // 使用密钥对消息进行签名，返回十六进制字符串
    Sign(message []byte, key string) ([]byte, error)
}
```